

MEASURING AND MITIGATING SOLUTION MODE COLLAPSE IN RLVR

Anonymous authors

Paper under double-blind review

ABSTRACT

A language model (LM) can usually answer the same question in more than one way, but reinforcement learning with verifiable rewards (RLVR) is indifferent to which correct answer a model produces. A solution will earn the same reward whether it is the thousandth copy of a familiar answer or one the model has never produced before. Yet, there is potential value in having the model retain multiple correct solutions as it is trained. For instance, multiple modes may give users a choice and provide problem-solving strategies that improve overall model performance. Here, we measure solution diversity through RLVR training using MODEBENCH, a set of multi-solution tasks, where the verifier returns both correctness and mode discovered. We find that RLVR post-training concentrates probability onto fewer correct modes even as accuracy holds or improves, and moreover, that frontier models are already highly concentrated. We then introduce Re:Max, which uses the verifier to index a memory containing one verified example per discovered mode and rehearses those modes uniformly. A solution found once is, therefore, practiced as often as one found repeatedly. Across three model scales, two RL objectives, and harder task constructions, Re:Max improves both how often a policy succeeds and how many different ways it can succeed.

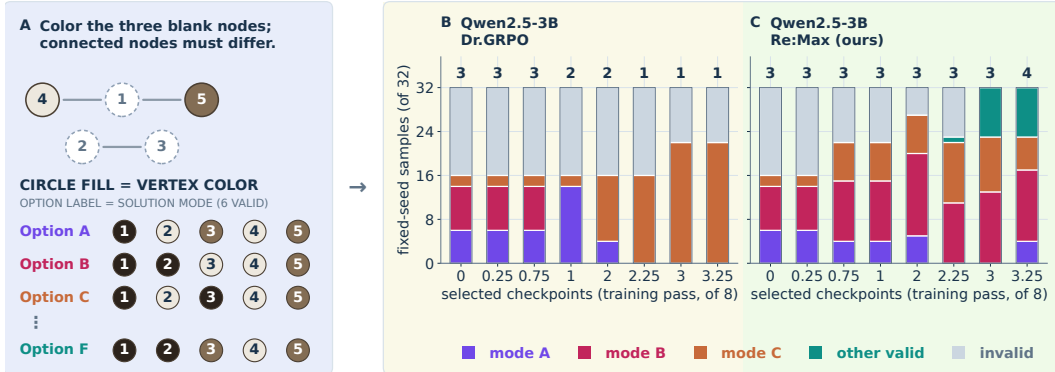


Figure 1: **Motivation.** Consider the Graph coloring problem shown in **A** (left), with six valid solutions. We evaluate Qwen2.5-3B across training, tracking the number and diversity of correct solutions. At each training pass, we show 32 sampled questions, eight from each of four matched runs. Training with Dr. GRPO (**B**) improves accuracy and leads to a single solution dominating the output. In contrast, Re:Max (**C**) increases both accuracy and diversity of correct solutions.

1 INTRODUCTION

Training and evaluation of reinforcement learning with verifiable rewards (RLVR) algorithms are driven entirely by model accuracy. However, what makes models useful at deployment depends on having different candidates to choose from, i.e. test-time search and reranking (Brown et al., 2024; Snell et al., 2024; Cobbe et al., 2021; Wang et al., 2023). This raises a question: as a model gets better at finding a correct answer during RLVR, *what happens to other correct candidates?*



ModeBench dataset and models



Code repository

Links anonymized for review.

RLVR trains against an executable check (Guo et al., 2025), but its binary reward does not distinguish a solution the model has produced a thousand times from one it has never produced before. Nothing in the objective asks the model to maintain diversity. Fig. 1 shows what this can look like on a graph-coloring problem with six valid answers: under Dr.GRPO (Liu et al., 2025), the policy becomes more accurate while its correct outputs concentrate onto a single mode.

Mode collapse, or narrowing of the output space, has been observed in alignment (Kirk et al., 2024; Cui et al., 2025), creative generation (Doshi & Hauser, 2024), brainstorming and ideation (Anderson et al., 2024), public argument (Kim et al., 2026), and LM-assisted writing (Abdulhai et al., 2026). Quantifying this narrowing requires deciding whether two outputs are actually the same answer. Token-level diversity (Li et al., 2016; Zhu et al., 2018) counts two spellings of one solution as variety, while a judge that groups answers by meaning uses a second model to evaluate whether a given response is distinct (Farquhar et al., 2024; Zheng et al., 2023).

Here, we investigate the prevalence and impact of mode collapse in problem-solving tasks under RLVR post-training. Our contributions are threefold:

1. **MODEBENCH**: We contribute a benchmark for empirically studying mode collapse in tasks allowing for multiple correct solutions, ranging from code evaluation to constrained planning. **MODEBENCH** provides both output correctness and maintains a unique identity for each available modal solution to a given problem. The benchmark spans five executable domains, each with five difficulty levels (Sec. 2).
2. We conduct comprehensive experiments on **MODEBENCH** across 4 open-weight model families and 7 frontier models. Across open-weight models and at 3 model scales, the number of solution modes collapses in every domain under both Dr.GRPO and GRPO. Further, in inference, all frontier models we measured reached fewer than 1.5 effective modes out of up to 355 unique solutions (Sec. 5.1 and App. N.1).
3. We propose *mode replay*, which stores one verified exemplar per mode and rehearses all modes equally. Added to Dr.GRPO (Liu et al., 2025), it raises accuracy and pairwise correct mode diversity across 3 model scales and all 5 **MODEBENCH** domains. We outperform 8 other RLVR baselines. Added to MaxRL (Tajwar et al., 2026) (which we dub as Re:Max), it adds diversity and accuracy across **MODEBENCH** difficulty levels (Sections 3 and 5.2–5.3).

Overall, our results quantify the severity of mode collapse and suggest that incorporating mode replay as part of RLVR post-training can lead to both increased diversity of solutions and improved model performance.

2 MODEBENCH: MEASURING SUCCESSFUL ALTERNATIVES

Building a benchmark to study solution model collapse is challenging for two reasons. First, each problem must allow for several valid discrete solutions. Second, given a model’s response, we must be able to determine which solution mode was reached. For example, in Fig. 2D, we show algebra problems in our benchmark. In this example, we show that the same algebraic solution can be reached through two different series of steps.

Ensuring these two properties hold is domain-specific. Fig. 2 illustrates a subset of solution modes across the considered domains; e.g., in **Countdown** and **MathIR**, modes mean the model reached

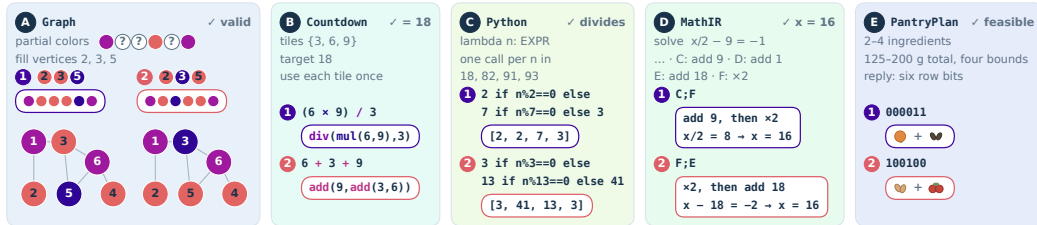


Figure 2: **MODEBENCH** comprises questions with many acceptable solutions across domains. (1) and (2) identify different solution modes across the five domains, which include Graph coloring, a normalized computation tree, executable Python code, algebraic problems and constraint optimization.

the same answer by a different route, whereas in Graph and PantryPlan, modes mean a different valid answer altogether. We provide the number of unique possible solutions per domain in Table 3. Refer to App. B and App. C for per-domain rules and prompts for dataset generation.

For each domain, we provide a verifier that evaluates whether a valid solution exists within the model’s response, as well as the mode a solution belongs to. This was built by creating in advance a comprehensive set of potential solutions to the problem. Precisely, for a prompt x with specification s_x , the validator maps an output y either to failure $\perp$ or to a key $V(y, s_x) \in \mathcal{C}_x$. Two accepted outputs are the same mode if and only if their keys agree. The full set of valid keys $\mathcal{C}_x$ is never revealed, neither its members nor its size. A policy, and the memory that stores its successes, only ever sees keys for answers the policy actually produced.

2.1 SUCCESS AND VERIFIED-MODE DIVERSITY

To measure performance on MODEBENCH, we must consider two distinct properties: how often the output is accurate, and, given that it is right, which of the available right answers has been given. A benchmark reporting only accuracy cannot capture the failure we study.

The obvious way to measure modal diversity is to count distinct correct answers in K samples. We define P as the chance a sample is correct and q_c be how that correct probability is divided among the valid answers:

$$\underbrace{\Pr[D_K > 0]}_{\text{pass@K}} = 1 - (1 - P)^K, \quad \underbrace{\mathbb{E}[D_K]}_{\text{distinct@K}} = \sum_c [1 - (1 - Pq_c)^K], \quad (1)$$

where D_K counts the distinct correct answers. However, both metrics increase with P , and thus a policy that is right more often may have higher `pass@K` and `distinct@K` while producing less diverse solutions.

We therefore propose an alternative metric, denoted as pairwise correct-mode diversity:

$$\text{PCMD} = 1 - \sum_c q_c^2 = \Pr[Z_1 \neq Z_2 \mid \text{both responses verified}], \quad (2)$$

measuring the joint probability of different solution modes given two correct responses. A PCMD of zero thus implies every correct answer belongs to the same solution mode, and an even spread over m yields a PCMD of $1 - 1/m$, e.g., if answers spread evenly across two possible solution modes, then the PCMD is $1/2$. We note that when infinite solution modes are possible, the maximum PCMD converges to 1, and that when the model outputs no correct solutions, we cannot calculate the metric. We only calculate PCMD when a model produces a minimum number of correct solutions; App. A.2 gives that threshold.

Fig. 3 shows how PCMD captures diversity of responses independently of accuracy. GPT-5.6 Sol has near-perfect `pass@8` for all five tasks considered, yet much weaker models in the Qwen2.5-Instruct family have higher PCMD. For the context of this work, we define a *solution mode collapse* as a decline in PCMD during RLVR.

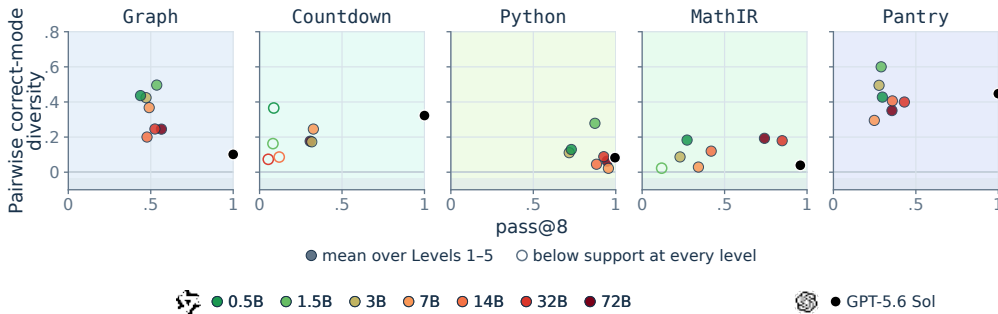


Figure 3: Higher performance is not associated with increased solution mode diversity. We show the relationship between a performance-related metric (`pass@8`; x -axis) and our proposed solution mode diversity metric (PCMD; y -axis) for GPT-5.6 Sol (in black), and Qwen2.5-Instruct models of various sizes (where parameter count is indicated by color). Note that accuracy and model size are not correlated with solution mode diversity.

2.2 HARDER TASKS WITH THE SAME NOTION OF A MODE

We construct MODEBENCH so that tasks can be made more or less challenging, which helps prevent $\text{pass}@8$ accuracy saturation. This allows us to explore the impact of problem difficulty on mode collapse. For each domain, we include 5 independent difficulty levels, e.g. for our Graph domain we allow more hidden vertices, and for Countdown we give the model more operands. Each difficulty level holds constant (1) our definition of mode, (2) our approximate distribution of modes, (3) our verifier constant, and (4) the sample size of 384 train and 128 test problems. To ensure (2) holds, and moreover, that the problems we sample are in fact more challenging, we fit each level with consideration to $\text{pass}@1$ and $\text{pass}@8$ of a reference model, e.g. Level 4 was tuned such that a Qwen2.5-Instruct-7B model has the same $\text{pass}@1$ and $\text{pass}@8$ scores as a Qwen2.5-Instruct-0.5B on Level 1.

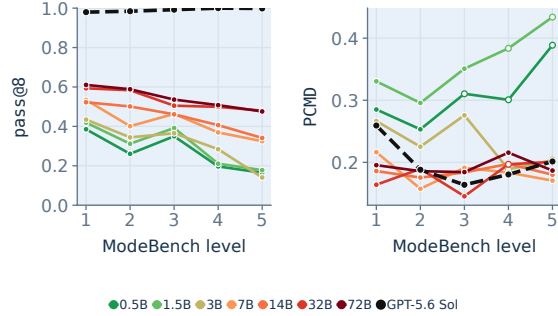


Figure 4: **Harder problems of the same domain exhibit fewer solution modes.** When we ask the Qwen2.5-Instruct family and GPT-5.6 Sol increasingly difficult questions from harder levels of MODEBENCH, $\text{pass}@8$ declined and so too does PCMD. For our benchmark, as problems get harder, solution mode collapse worsens.

LMs with a higher parameter count generally have higher $\text{pass}@8$ and can solve more problems. Yet, in Fig. 4, for our harder problems, we find that solution mode diversity declines, even as our underlying mode count per level remains stable. This is most noticeable when frontier models are evaluated. Even though correctness is saturated, we are able to use PCMD to isolate solution mode collapse (App. N.1), and this phenomena happens across MODEBENCH. For example, frontier models answer .991 of questions from Level 1-3 correctly under normalized grading, but their mean PCMD is only .232, below the .570 of the most diverse sub-2B model, which answers .273 of its prompts.

2.3 CONCENTRATION IN FRONTIER MODELS

A common fix for solution mode collapse is to increase the sampling budget, as repeated sampling can substantially increase solution coverage at test time (Brown et al., 2024; Yue et al., 2025). But, across MODEBENCH, we find that more samples do not recover the complete set of modes. For example, all 1,024 GPT-5.6 Sol draws on the 128 Level-3 Graph problems verify, yet eight draws yield just 1.16 distinct colorings, and 95.1% of correct pairs repeat one (App. N.1). Fig. 5 follows MODEBENCH at three levels out to 512 draws per problem, and the curves flatten far below the modal set: an LM that has settled on two colorings does not find the rest even after being asked five hundred times.

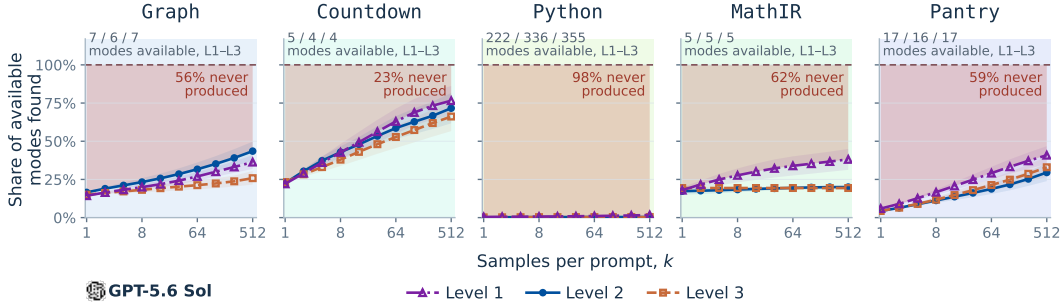


Figure 5: **Asking the model more times may not recover all available modes.** Our other frontier measurements ask each question eight times, so we test directly whether a larger budget recovers the missing modes. GPT-5.6 Sol, with medium reasoning on, answers 32 problems per domain and level with 512 responses each. Curves are distinct verified modes in random k -subsets as a share of the modes those problems admit, and the shaded band is what no level ever produced. Shading is the pointwise 95% problem bootstrap. Printed counts are the enumerated support those problems admit, in Level 1/2/3 order, and every panel shares one scale (App. K.3).

Python is the extreme case. Its problems admit several hundred distinct return vectors on average, and five hundred draws surface between one and four solutions, under two percent of all possible solutions, and under one percent at Levels 2 and 3. Each doubling of the sampling budget buys a fraction of one mode, against gaps of several modes everywhere and several hundred in Python. This problem becomes worse as we increase problem difficulty. All 7 frontier models tested narrow in PCMD from Level 1 to Level 3 while staying near-perfect on correctness (Fig. 15, App. N.1).

We similarly check our local models over larger samplings, and we find quadrupling to $K = 32$ draws per problem moves control PCMD by at most .010, and MathIR at $K = 64$ draws is as concentrated as at $K = 8$ (Apps. J and K), while disabling reasoning lowers correctness and verified-mode counts in every deployment tested, though its PCMD effect differs by deployment (Table 59, App. O.6).

We do, however, note that cheap inference interventions do exist. For example, we find that because different models collapse onto different answers, mixing responses from two independent models covers more verified solution modes than either alone (App. L.2). Additionally, raising inference temperature gives more solution modes too, but at the cost of model accuracy (Fig. 39).

3 RE:MAX: LEARN FROM NEW SAMPLES AND REMEMBER SUCCESSES

3.1 THEORETICAL INTUITION FOR MODE COLLAPSE

Solution mode collapse follows from two properties of the objective: 1) the binary verifier assigns the same reward to every correct answer, so the objective provides no incentive to preserve one correct mode over another; and 2) an on-policy update can reinforce only the modes that appear in its fresh rollout samples, which are drawn according to the policy’s current preferences. Together, these properties create a positive feedback loop, where a mode with even a small early advantage is sampled more often, receives more reinforcement, and therefore becomes even more likely to be sampled in subsequent updates. Figure 6 illustrates this process.

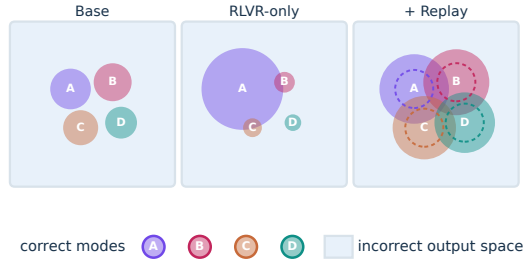


Figure 6: **RLVR early "winner" (A) becomes the predominant way to solve.** Under base model, we see that before RLVR, many modes exist. RLVR-only training concentrates on one mode. Our replay approach keeps every buffered mode & leaves room for more.

We formalize this intuition in three theoretical results, developed in detail in the appendices. First, Theorem P.4 shows that, under an independent-logit parametrization and Euclidean expected-gradient flow, this positive feedback leads to a “winner-takes-all” limit: a unique initial leader among correct modes eventually dominates, as Sinha et al. (2026) and Lochab et al. (2026) also establish. Second, App. P.14 shows that this result depends on the interaction between binary rewards and optimization geometry; binary rewards alone do not force this concentration. Third, Lemma P.39 characterizes a limitation of standard RLVR; LMs post-trained with RLVR can amplify rare modes when sampled, but both the chance of sampling a mode and its expected absolute logit-gradient contribution vanish as its probability approaches zero. When the applied advantages sum to zero within the group, a mode absent from the current batch has no gradient.

3.2 REPLAYING DISTINCT VERIFIED SUCCESSES

Our theoretical analysis suggests that, to achieve diversity in inference we need to maintain our solution modes during training. We address this challenge drawing inspiration from classic RL approaches towards memory management through replay (Lin, 1992; Mnih et al., 2013; Rolnick et al., 2019). Once a successful solution mode has been found and stored, replay allows us to continue training on it whether or not the current policy rediscovers it. Under a fixed-buffer analysis, a constant replay from the existing set of unique found modes keeps every buffered mode above a positive probability floor (Theorem P.22).

For each training prompt x , we maintain an initially empty, bounded buffer $\mathcal{B}_x$. The first time a verified mode appears, we store one exemplar e_c that realizes it (subsequent rediscoveries do not modify the buffer). A deterministic schedule revisits nonempty buffer throughout training. For an exemplar of token length $|e_c|$, the replay loss is

$$L_{\text{replay}}(x) = -\frac{1}{|\mathcal{B}_x|} \sum_{c \in \mathcal{B}_x} \frac{\log \pi_{\theta}(e_c | x)}{|e_c|}. \quad (3)$$

Uniform weighting breaks the link between discovery frequency, how often a mode appears in fresh rollouts, and rehearsal frequency, how often a stored mode is replayed. Once a mode enters the buffer, it is rehearsed as often as every other retained mode, so frequently sampled modes receive no additional replay.

Adding this loss to the fresh update from standard MaxRL (Tajwar et al., 2026) gives our primary method, Re:Max. But, this principle is modular, and can be applied to any RLVR algorithm. For example, we also evaluate with Dr.GRPO (Liu et al., 2025) resulting in the algorithm, Re:Dr. Solution modes discovered during training do not enter the on-policy reward group, and contribute only through the separate replay loss. This separation allows one to vary the RLVR objective and the replay mechanism independently. Finally, our replay buffers contain only responses to training prompts; held-out evaluation therefore measures changes in the policy itself rather than retrieval or recitation from the buffer (App. D).

Fig. 7 illustrates how replay changes learning dynamics. In the standard RLVR setting, a policy learns only from modes present in the current sampling, which often leads previously discovered modes to disappear. Re:Max adds an additional incentive for the policy to continuously generate previously found solution modes that yield correct solutions. We note that this is fundamentally different from approaches that encourage diversity by modifying the on-policy learning signal to favor rare or diverse outcomes (Song et al., 2025; Li et al., 2026a), as we do not use mode identity to modify the task reward or advantage.

4 EXPERIMENTAL DESIGN

In our main experiment, we examine whether replay improves correctness and solution-mode diversity across different RLVR objectives. We compare the impact of our replay component against both Dr.GRPO (Liu et al., 2025) and MaxRL (Tajwar et al., 2026). First, on all five MODEBENCH Level 1 train/test sets we carry out five seeds of post-training using Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B. We train for eight epochs with $G = 16$ new rollouts. We note that replay does not increase this sampling budget. Instead, it performs an additional training update using at most $M = 16$ previously collected exemplars from the replay buffer for each visited prompt (Table 6). Second, we conduct the same experiment on MODEBENCH Levels 2 and 3 using only a Qwen2.5-0.5B. Third, we compare our method with other approaches, including frequency-weighted replay, UCPO (Lochab et al., 2026), RLEP (Zhang et al., 2025), GAPO (Anschel et al., 2025), SetPO (Li et al., 2026a), and fixed Semantic-MaxEnt under a specific setting (Qwen2.5-0.5B, five seed, level 1).

We additionally evaluate 7 hosted models: GPT-5.6 Sol (OpenAI, 2026b), GPT-5.4 (OpenAI, 2026a), Grok 4.3 (xAI, 2026), Kimi K3 (Moonshot AI, 2026), DeepSeek V4 Pro (DeepSeek-AI, 2026), Claude

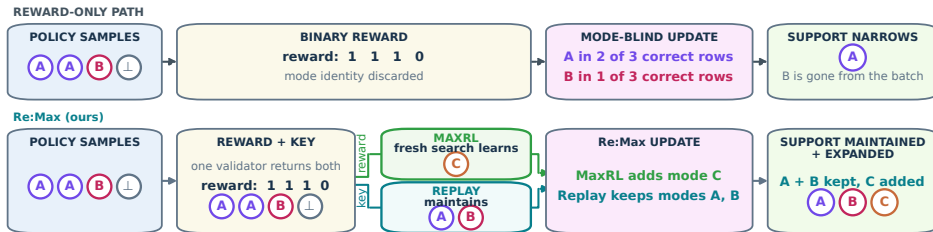


Figure 7: **Replay preserves successes that on-policy learning may stop seeing.** MaxRL learns from the policy’s current samples, while replay stores one verified exemplar per discovered mode and revisits those exemplars with equal weight. Together, these two learning paths allows Re:Max to preserve previously discovered solution modes while continuing to explore for new ones.

Table 1: **Compared objectives under Qwen2.5-0.5B, MODEBENCH Level 1.** Each method is compared against the same task performed under Dr.GRPO; avg weights the five domains equally within seed (5-seeds). Bold marks a column’s best. An em-dash means that there are few more-than-one solution pairs to evaluate, and a superscript gives the paired seed count where it falls below five. Red-to-green shading scales to each column’s largest magnitude.

	Δ pass@8						Δ PCMD					
	Count			Pantry			Count			Pantry		
	Graph	down	Python	MathIR	Plan	Avg	Graph	down	Python	MathIR	Plan	Avg
Re:Dr (<i>ours</i>)	+0.646	+0.191	+0.356	+0.285	+0.207	+0.337	+0.559	+0.480	+0.082	+0.017	+0.334	+0.294
Re:Max (<i>ours</i>)	+0.621	+0.185	+0.509	+0.277	+0.199	+0.358	+0.525	+0.504	+0.249	+0.005	+0.317	+0.320
GAPO	+0.175	+0.011	+0.009	−0.054	+0.279	+0.084	+0.131	+0.340	.000	.000	+0.653	+0.225
SetPO	+0.064	−0.050	+0.497	+0.051	+0.154	+0.143	+0.039	+0.007	+0.013	+0.003	+0.266	+0.066
MaxRL (no replay)	+0.213	+0.101	.000	−0.066	+0.009	+0.051	+0.086	+0.015	.000	.000	.000	+0.020
RLEP-Dr	+0.098	+0.103	.000	+0.126	.000	+0.066	+0.035	+0.012	.000	−0.001	.000	+0.009
UCPO	−0.003	+0.133	.000	+0.031	+0.053	+0.043	−0.003 ²	+0.019	.000 ²	−0.001	.000 ²	+0.009
Semantic-MaxEnt	+0.002	+0.141	.000	−0.121	+0.201	+0.045	+0.004	+0.020	.000	+0.001	—	+0.006
GRPO	+0.037	+0.159	.000	−0.052	+0.023	+0.034	+0.008	+0.001	.000	+0.001	.000	+0.002
Untrained checkpoint	+0.132	−0.383	−0.172	−0.292	+0.380	−0.067	+0.108	—	—	+0.027	+0.591	+0.242

Opus 5 (Anthropic, 2026b), and Claude Opus 4.8 (Anthropic, 2026a) on MODEBENCH Levels 1–3, collecting eight responses per prompt and grading them through a fixed formatting normalizer so that presentation differences are not counted as verified alternatives (App. O). We carry out the same inference-only evaluation Level 1–5 across the available scales and benchmark levels of SmolLM2 (Ben Allal et al., 2025), Qwen2.5 (Qwen et al., 2024), Falcon3 (Falcon-LLM Team, 2024), and OLMo-2 (Team OLMo et al., 2024). These experiments are inference-only; no replay (or training) is applied.

5 RESULTS

Our results are three-fold. First, we document solution-mode collapse under RLVR on MODEBENCH (Section 5.1); Second, we find that replay reverses this collapse without sacrificing correctness (Section 5.2), outperforming 8 existing RLVR approaches. Third, we show that replay gains persist across difficulty levels (Section 5.3), KL-chaining, and domain changes.

5.1 RLVR CONCENTRATES SOLUTION MODES

RLVR replaces diverse correct attempts with the same repeated answers. We compare each policy’s solution-mode distribution before and after training using PCMD. Across Dr.GRPO, GRPO, and MaxRL results, we find fewer solution modes after RLVR post-training in 14 out of 15 domains for Qwen2.5-3B (Fig. 8), with the same pattern appearing across model scales and difficulty levels (Fig. 27). The starkest case is Python, where Qwen2.5-3B goes from solving 73 of 128 held-out prompts in multiple ways to as few as 22 after Dr.GRPO post-training.

As outputs concentrate, rollouts under RLVR lose training signal. Dr.GRPO and MaxRL learn from a rollout group only when it contains a mix of correct and incorrect responses. Thus, we audit every rollout group during training to track how often they have provided a non-zero learning signal. We find that Re:Max systematically increases the percentage of roll-

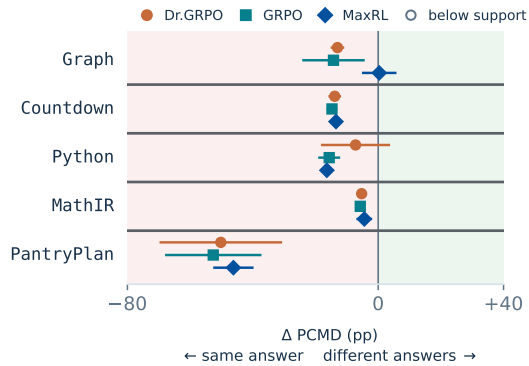


Figure 8: **Training reduces solution mode diversity.** We compare PCMD before and after RLVR on Qwen2.5-3B, over MODEBENCH Level 1 test set, under three objectives, five domains, five seeds. The figure shows the change in PCMD from before to after. Marks are means over training seeds with nominal 95% intervals. In 14 of 15 comparisons, we find fewer solution modes after training. See Fig. 27 for other model scales.

out groups with a learning signal (App. H.4); e.g., on Python, 18.5% of groups have successes and failures under replay, compared to only 1.5% without.

5.2 REPLAY IMPROVES BOTH ACCURACY AND DIVERSITY

Replay improves accuracy and diversity across RLVR algorithms and model scales. We add our proposed verified-mode replay mechanism to Dr.GRPO and MaxRL and compare each against its matched no-replay control across five MODEBENCH domains and three model scales. Across these comparisons, we find that Re:Max and Re:Dr. (ours) shift the outcome toward both higher pass@8 and greater solution-mode diversity (Fig. 9; Tab. 8).

Mode diversity comes from retaining modes; balancing makes it stronger. A single-slot buffer, rehearsing one verified success per prompt and never seeing a second key, already recovers 58% of Re:Dr’s diversity effect (App. Q.4). To isolate balancing itself, we compare uniform replay with a frequency-weighted variant that rehearses modes in proportion to their discovery rate. Holding buffer construction, advantage scheduling, and loss update unchanged, uniform weighting preserves greater verified diversity, most clearly in Graph, Countdown, and PantryPlan: in Graph it recovers roughly 0.6 additional modes per prompt while raising pass@8 by 6.6 points. Across domains, these gains come without an aggregate difference in correctness (App. F.7, App. Fig. 25).

Alternative diversity mechanisms produce isolated rather than consistent gains. We compare Re:Max and Re:Dr(ours) against UCPO (Lochab et al., 2026), RLEP (Zhang et al., 2025), GAPO (Anschel et al., 2025), SetPO (Li et al., 2026a), and fixed Semantic-MaxEnt (Tab. 1, App. F.5). Some of these methods produce substantial gains in individual domains, e.g., GAPO reaches $2.9\times$ its control’s effective number of verified modes in PantryPlan, while SetPO raises pass@8 by 147% in Python. Yet these gains are domain-specific, while Re:Max improves both metrics across domains, raising pass@8 by 90% relative to Dr.GRPO and reaching $1.9\times$ its effective number of verified modes (App. P.12).

5.3 REPLAY REMAINS EFFECTIVE BEYOND THE BASE TRAINING SETTING

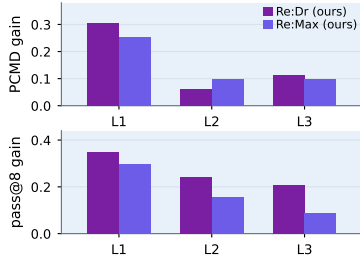


Figure 10: **Replay gains persist across difficulty levels.** Bars show the gain from adding replay to Dr.GRPO (Re:Dr) and MaxRL (Re:Max) at each MODEBENCH level, averaged across domains for Qwen2.5-0.5B. Replay improves both PCMD (top) and pass@8 (bottom) at all three levels.

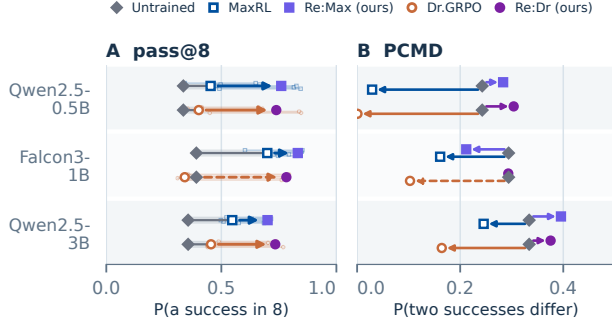


Figure 9: **Replay leads to higher accuracy and solution mode diversity.** Diamonds are initial LMs; open/filled markers are training without/with replay. (A) averages all five domains within seed; faint paths show seeds. (B) draws both arrows from the untrained model, one to the standard RLVR objective alone, one to it with replay, over the domains where every arm clears the support bar. App. Fig. 18 gives results for each domain and scale.

Replay’s gains persist on harder task constructions. We repeat the replay-versus-control comparison within each of the first three MODEBENCH levels, holding the level and RLVR objective fixed. Re:Dr. and Re:Max (ours) improves average pass@8 and PCMD under both Dr.GRPO and MaxRL at every level (Fig. 10). At Level 2, for instance, Graph rises from 1.04 to 1.55 effective verified modes under Dr.GRPO, and from 1.13 to 1.48 under MaxRL. Gains vary across domains, but remain positive in aggregate even at Level 3 (App. F.3).

Replay leaves more usable alternative correct solutions when the task changes. We ask whether Re:Max / Re:Dr has a downstream impact on inference. Ideally, replay should make a model more

able to give a correct answer when the original most-common answer is no longer available. To test, we take a portfolio of eight generated answers, remove one option from the task—e.g. an ingredient in `PantryPlan` or a color in `Graph`—and re-check under the modified task. Across all 5 domains, Re:Max / Re:Dr (ours) are more likely to give a valid alternate answer. When no answer is valid, Re:Max / Re:Dr find a new valid answer in fewer additional attempts than RLVR or tuned-temperature sampling (Apps. M.3, L.1).

Reference KL and replay preserve different sources of diversity. One explanation for the diversity gains of Re:Max / Re:Dr (ours) is that the model retains the diversity of its base. We test this by anchoring the policy to its base: we sweep the coefficient of a KL penalty to the initial policy and compare its correctness–diversity curve with Re:Max / Re:Dr (ours). The anchor is limited by the diversity already present in its reference, whereas Re:Max/ Re:Dr (ours) is limited by the modes its buffer discovers. Averaged over the three domains with a reportable initial PCMD, matching diversity with a KL anchor costs model correctness, although in `Python` and `Pantry` the reference is already broad enough for the anchor to win (Fig. 11). We further show analytically that diversity depends on whether all modes were contained within the initial policy or discovered during training (App. P.11).

6 RELATED WORK

Measuring solution diversity under alignment RLHF and reasoning RL can reduce output diversity, entropy, and high-budget coverage (Kirk et al., 2024; Dang et al., 2025; Cui et al., 2025; Yue et al., 2025). Measuring this collapse, however, requires defining when two correct answers differ meaningfully. Semantic clustering infers it from language (Farquhar et al., 2024), which is general but depends on a judge. MODEBENCH instead defines solution identity through the same executable check that assigns reward, so two outputs differ exactly when they execute differently: a narrower but cheaper measure, shared across training and evaluation. Preserving alternatives matters because sampling-based selection relies on generating multiple viable proposals (Brown et al., 2024; Snell et al., 2024; Cobbe et al., 2021; Lightman et al., 2024).

Expanding diversity. Outcome bonuses, set prediction, distribution matching, and uniform-correct objectives all reshape the learning signal (Song et al., 2025; Chen et al., 2025; Li et al., 2025; 2026a;b; Lochab et al., 2026), while MaxRL combines objectives across sampling budgets (Tajwar et al., 2026). Yet existing approaches still depend on what the current policy samples—precisely where our analysis finds that solution diversity disappears. Memory can break this dependence: standard replay (Lin, 1992; Rolnick et al., 2019), self-imitation learning (Oh et al., 2018), and RLEP (Zhang et al., 2025) all revisit past successes in proportion to how often they occur. Instead, we homogeneously balance replay across solution modes, an idea related to explicitly preserving behavioral diversity in quality-diversity methods (Mouret & Clune, 2015; Ecoffet et al., 2021).

7 CONCLUSION

We contribute MODEBENCH, a benchmark for studying solution mode diversity in RLVR, and Re:Max, a replay-based method that preserves discovered solution modes during RLVR. Across objectives, model scales, and task difficulties, replay retains solution diversity while improving correctness. Diversity may itself be a target for optimization: future work can move beyond preserving discovered modes towards training models to produce desired distributions over ways to solve a problem.

Limitations. Replay retains only what sampling finds, and mode count says nothing about the “worth” of additional solutions. Our equivalences are executable by construction, so future work is needed to evaluate the benefit and feasibility of replay in more open-ended solution spaces.

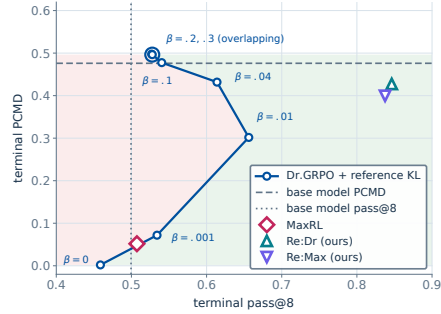


Figure 11: **Replay and KL anchoring preserve different diversity.** Means over the 3 domains with reportable initial PCMD, Qwen2.5-0.5B. The anchor curve sweeps β ; larger β preserves more diversity at lower pass@8. Red/green marks lower/higher pass@8 than KL at equal measured diversity (App. P.12).